AGENTIC AI & FULL STACK INTERVIEW PREPARATION GUIDE

500 Questions · 10 Topics · Easy → Medium → Hard

RAG · Vector DBs · LangChain · LangGraph · CrewAI · Prompt Engineering

LLM APIs · Python Async · DynamoDB · Docker · Full Stack

Prepared for Kolla Charan Teja

Target: SDE-1 / AI Engineer / Agentic AI Full Stack Roles

How to Use This Guide

■ **Structure:** 10 topics × 50 questions each = 500 total. Each topic has 20 Easy, 20 Medium, and 10 Hard questions.

■ **Easy questions:** Conceptual definitions and basics. You should be able to answer all of these before applying.

■ **Medium questions:** Practical implementation and design. These appear most in 1-3 year experience interviews.

■ **Hard questions:** System design and deep-dive. Aim to answer 6-7 of 10 for senior/specialist roles.

■ **Resources:** Each topic ends with curated free resources. Prioritise official docs + DeepLearning.AI courses.

■ **Study Plan:** Week 1-2: RAG + Vector DBs. Week 3-4: LangChain/LangGraph. Week 5-6: CrewAI + LLM APIs. Week 7: Prompt Eng + Python. Week 8: Docker + DynamoDB + Full Stack review.

■ **Priority Order:** For the JD you shared: RAG > LangGraph > LLM APIs > Prompt Eng > Python Async > Docker > DynamoDB.

8-Week Learning Roadmap

Week 1 — RAG Foundations

- ■ Day 1-2: Read 'RAG from Scratch' LangChain YouTube playlist (first 8 videos)
- ■ Day 3-4: Build a PDF Q&A; app — PyPDF + LangChain + ChromaDB + FastAPI
- ■ Day 5-6: Add metadata filtering, implement hybrid search
- ■ Day 7: Answer all 50 RAG questions in this guide (write answers, don't just read)

Week 2 — Vector Databases & Embeddings

- ■ Day 1-2: DeepLearning.AI 'Vector Databases: from Embeddings to Applications' course
- ■ Day 3: Compare FAISS vs ChromaDB — switch your Week 1 project to FAISS
- ■ Day 4-5: Set up Pinecone free tier, migrate your app to Pinecone
- ■ Day 6-7: Benchmark 3 embedding models on your own corpus

Week 3 — LangChain + LangGraph

- ■ Day 1-2: DeepLearning.AI 'AI Agents in LangGraph' course (2hrs, free)
- ■ Day 3-4: Build a ReAct agent with 3 custom tools using LangGraph
- ■ Day 5-6: Add human-in-the-loop and memory to your LangGraph agent
- ■ Day 7: Set up LangSmith tracing for your agent — trace every run

Week 4 — CrewAI + Multi-Agent Systems

- ■ Day 1-2: DeepLearning.AI 'Multi AI Agent Systems with crewAI' course (free)
- ■ Day 3-4: Build a 3-agent crew: Researcher + Writer + Fact Checker
- ■ Day 5-6: Combine CrewAI with your RAG system as a tool
- ■ Day 7: Answer all 50 multi-agent questions in this guide

Week 5 — LLM APIs + Prompt Engineering

- ■ Day 1-2: DeepLearning.AI 'Prompt Engineering for Developers' (1hr, free)
- ■ Day 3: Implement function calling with OpenAI + tool use with Anthropic
- ■ Day 4-5: Build a cost tracker — log every API call with tokens + cost
- ■ Day 6-7: Implement semantic caching to reduce API calls by 40%+

Week 6 — Python Async + FastAPI

- ■ Day 1-2: Build full async FastAPI backend with streaming LLM endpoint
- ■ Day 3: Add rate limiting, retry logic, and error handling
- ■ Day 4-5: Write pytest tests for every endpoint, mock LLM calls
- ■ Day 6-7: Add WebSocket endpoint for real-time agent progress updates

Week 7 — DynamoDB + Docker

- ■ Day 1-2: YouTube 'DynamoDB Full Guide' — set up DynamoDB Local
- ■ Day 3: Design and implement chat history schema in DynamoDB with TTL
- ■ Day 4: YouTube 'Docker Tutorial' by TechWorld with Nana
- ■ Day 5-6: Dockerize your entire AI app with docker-compose
- ■ Day 7: Multi-container setup: FastAPI + ChromaDB + Redis

Week 8 — Integration + Portfolio Project

■ Day 1-3: Build final portfolio project combining everything (see below)

■ Day 4-5: Deploy on AWS (EC2 or ECS) or Railway/Render

■ Day 6: Record a 3-min demo video for GitHub README

■ Day 7: Update resume with new skills + apply to 20 companies

■ Final Portfolio Project (Week 8)

Build: 'AI Research Assistant' — A full-stack agentic app where a user uploads documents, asks questions (RAG), and triggers a multi-agent workflow (LangGraph) that researches the web, analyses documents, and produces a structured report. Stack: Next.js + FastAPI + LangGraph + ChromaDB + DynamoDB (session storage) + Docker Compose. This single project covers every skill in both JDs you shared.

RAG — Retrieval-Augmented Generation

Foundations, Architecture, Pipelines & Production

■ EASY

Easy Questions — Conceptual Foundations

1. What does RAG stand for and what problem does it solve?
2. What are the two main components of a RAG system?
3. Why can't we just rely on an LLM's training data alone for answering factual questions?
4. What is a 'knowledge base' in the context of RAG?
5. What is the role of the 'retriever' in a RAG pipeline?
6. What is the role of the 'generator' in a RAG pipeline?
7. What is document chunking and why is it needed?
8. What is an embedding in simple terms?
9. What does 'semantic search' mean compared to keyword search?
10. What is a vector database and how does it differ from a SQL database?
11. Name 4 popular vector databases used in RAG systems.
12. What is cosine similarity and why is it used in RAG retrieval?
13. What is 'context window' in an LLM and why does it matter for RAG?
14. What is the difference between dense retrieval and sparse retrieval?
15. What is BM25 and when would you use it?
16. What is a 'prompt template' in a RAG system?
17. What is the difference between RAG and fine-tuning?
18. What does 'top-k retrieval' mean?
19. What is document ingestion in a RAG pipeline?
20. What is FAISS and what does it stand for?

MEDIUM

Medium Questions — Implementation & Design

1. Explain the end-to-end flow of a RAG pipeline from document ingestion to final answer.
2. What is chunk overlap and why is it important? What are common chunk sizes?
3. Compare sentence-level, paragraph-level, and fixed-size chunking strategies.
4. What is Hybrid Search in RAG? How do you combine BM25 and dense retrieval?
5. Explain Reciprocal Rank Fusion (RRF) for combining retrieval scores.
6. What is a re-ranker and when should you add one to your RAG pipeline?
7. What is the difference between FAISS flat index and HNSW index?
8. What are the trade-offs of using Pinecone vs Chroma vs FAISS?
9. How do you handle multi-turn conversations in a RAG system?
10. What is metadata filtering in vector search? Give a practical example.

11. What is the 'lost in the middle' problem in RAG and how do you mitigate it?
12. How would you implement RAG using LangChain? Name the key classes involved.
13. What is a MultiQueryRetriever and when is it useful?
14. Explain Parent-Child chunking strategy and its advantages.
15. What is contextual compression in RAG retrieval?
16. How do you evaluate RAG system quality? What metrics would you use?
17. What is RAGAS? Name the 4 key metrics it measures.
18. What is hallucination in RAG and how do you reduce it?
19. How would you handle PDF parsing for RAG — what are common challenges?
20. What is an ensemble retriever and how does it work?

■ HARD

Hard Questions — System Design & Deep Dives

1. Design a production-grade RAG system for a 10,000-page legal document corpus. Walk through every architectural decision.
2. Explain the full mathematical basis of how approximate nearest neighbor (ANN) search works in HNSW.
3. What is Hypothetical Document Embedding (HyDE) and how does it improve retrieval quality?
4. Design a RAG evaluation framework from scratch without using RAGAS. What would you measure?
5. How would you implement adaptive chunking that adjusts chunk size based on content structure?
6. What is agentic RAG and how does it differ from naive RAG?
7. You have a RAG system with 70% precision but low recall. What specific steps would you take to improve recall?
8. Explain Corrective RAG (CRAG) — what problem does it solve and how does it work?
9. How do you implement multi-modal RAG that handles both text and images?
10. What is Self-RAG and how does it use reflection tokens to improve output quality?
11. Design a real-time RAG pipeline that can handle documents being added continuously without full re-indexing.

■ Study Resources

Official Docs:

→ LangChain RAG Tutorial — python.langchain.com/docs/tutorials/rag

→ LlamaIndex RAG Guide — docs.llamaindex.ai

→ FAISS Documentation — faiss.wiki.github.io

Best Videos/Courses:

→ YouTube: 'RAG from Scratch' by LangChain (full playlist, 20 videos)

→ DeepLearning.AI: 'Building and Evaluating Advanced RAG' (free, 1hr)

→ YouTube: 'Advanced RAG Techniques' by Sam Witteveen

Papers to Read:

→ Original RAG Paper: 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks' (Lewis et al. 2020)

→ Self-RAG Paper (2023) — arxiv.org/abs/2310.11511

→ RAGAS Paper — arxiv.org/abs/2309.15217

Practice Projects:

→ Build: PDF Q&A; bot using LangChain + ChromaDB + FastAPI

→ Build: Multi-document RAG with metadata filtering

→ Build: RAG with evaluation using RAGAS

Vector Databases & Embeddings

FAISS · Chroma · Pinecone · Weaviate · Embeddings

■ EASY

Easy Questions — Conceptual Foundations

1. What is a vector and how does it represent text meaning?
2. What is an embedding model and what does it output?
3. What is the difference between word embeddings and sentence embeddings?
4. Name 3 popular open-source embedding models.
5. What is OpenAI's text-embedding-3-small model and when to use it?
6. What is cosine similarity? How is it calculated?
7. What is Euclidean distance in vector space?
8. What is a vector index and why is it needed?
9. What does FAISS stand for? Who built it?
10. What is Chroma DB and how do you install it?
11. What is Pinecone and how is it different from FAISS?
12. What is a collection in ChromaDB?
13. What is a namespace in Pinecone?
14. What is the difference between L2 distance and cosine distance?
15. What are the basic CRUD operations in a vector database?
16. What is dimension in an embedding vector? (e.g., 768, 1536)
17. What is batch embedding and why is it used?
18. What is metadata in vector databases? Give 3 examples.
19. What is upsert in Pinecone?
20. What is a flat index in FAISS?

MEDIUM

Medium Questions — Implementation & Design

1. Explain the difference between FAISS IndexFlatL2, IndexFlatIP, and IndexIVFFlat.
2. What is HNSW (Hierarchical Navigable Small World) and why is it preferred for production?
3. What is IVF (Inverted File Index) in FAISS and how does it speed up search?
4. Compare Pinecone vs Weaviate vs Chroma for a production use case with 10M vectors.
5. How do you persist a ChromaDB collection to disk and reload it?
6. Explain the trade-off between recall and speed in approximate nearest neighbor search.
7. What is quantization in vector indexes? What is PQ (Product Quantization)?
8. How does Weaviate support hybrid search (vector + keyword)?
9. How do you handle versioning of embeddings when you change your embedding model?
10. What is a semantic cache and how does it reduce LLM API costs?

11. How would you implement metadata filtering in FAISS (which natively doesn't support it)?
12. What is the 'curse of dimensionality' and how does it affect vector search?
13. Explain how you would scale a Chroma instance for 1 million documents.
14. What is sentence-transformers library and how do you use it for embedding?
15. Compare all-MiniLM-L6-v2 vs text-embedding-ada-002 vs text-embedding-3-large.
16. What is dot product similarity vs cosine similarity — when does it matter?
17. How do you handle multilingual embeddings in a RAG system?
18. What is an embedding fine-tuning and when is it worth doing?
19. Explain the concept of 'embedding drift' and why it's a production concern.
20. How do you benchmark the quality of an embedding model for your specific domain?

■ HARD

Hard Questions — System Design & Deep Dives

1. Design a vector database architecture that serves 100M vectors with sub-50ms p99 latency.
2. Explain how to implement your own in-memory vector store from scratch using Python and numpy.
3. How does Weaviate's HNSW implementation differ from a pure in-memory HNSW?
4. What is matryoshka embedding and how does it allow variable-dimension retrieval?
5. Design a multi-tenant vector database architecture where each user's data is isolated.
6. Explain the math behind Product Quantization and how it reduces memory usage.
7. You're getting poor retrieval quality despite good embeddings. What are 10 things you'd investigate?
8. What is ColBERT and how does late interaction differ from bi-encoder retrieval?
9. How would you implement a real-time vector index that supports inserts without full re-indexing?
10. Compare the consistency models of Pinecone, Weaviate, and Qdrant for distributed deployments.
11. Design an embedding pipeline for a multilingual document corpus of 5 languages.

■ Study Resources

Official Docs:

→ ChromaDB Docs — docs.trychroma.com

→ FAISS Wiki — github.com/facebookresearch/faiss/wiki

→ Pinecone Docs — docs.pinecone.io

→ Weaviate Docs — weaviate.io/developers/weaviate

Best Videos/Courses:

→ YouTube: 'Vector Databases Explained' by Fireship

→ DeepLearning.AI: 'Vector Databases: from Embeddings to Applications' (free)

→ YouTube: 'FAISS Tutorial' by James Briggs

Practice Projects:

→ Build: Semantic search over Wikipedia using FAISS + sentence-transformers

→ Build: Switch from FAISS to ChromaDB in an existing RAG app

→ Build: Benchmark 3 embedding models on the same corpus

LangChain & LangGraph

Chains · Agents · Memory · Tools · Graphs · State Machines

■ EASY

Easy Questions — Conceptual Foundations

1. What is LangChain and what problem does it solve?
2. What is an LLMChain in LangChain?
3. What is a PromptTemplate in LangChain?
4. What is a Runnable in LangChain v0.2+?
5. What is LCEL (LangChain Expression Language)?
6. What is a LangChain Tool?
7. What is an Agent in LangChain?
8. What is Memory in LangChain? Name 3 memory types.
9. What is ConversationBufferMemory?
10. What is a Document Loader? Give 3 examples.
11. What is a TextSplitter? Name 2 common ones.
12. What is a VectorStore in LangChain?
13. What is a Retriever in LangChain?
14. What is RetrievalQA chain?
15. What is the difference between a chain and an agent in LangChain?
16. What is LangSmith and what is it used for?
17. What is a callback in LangChain?
18. What is ChatOpenAI vs OpenAI in LangChain?
19. What is a BaseChatModel?
20. What is OutputParser in LangChain?

MEDIUM

Medium Questions — Implementation & Design

1. Explain LCEL pipe syntax (|) with a complete working example.
2. What is the difference between .invoke(), .stream(), and .batch() in LangChain?
3. How do you implement a RAG chain using LCEL from scratch?
4. What is MapReduceDocumentsChain and when is it used?
5. What is the difference between create_react_agent and create_openai_tools_agent?
6. How does ReAct (Reasoning + Acting) loop work in an agent?
7. What is LangGraph and how does it differ from LangChain agents?
8. What is a StateGraph in LangGraph? What is the State object?
9. Explain nodes and edges in LangGraph with a concrete example.
10. What is a conditional edge in LangGraph and how do you use it?

11. How do you implement a human-in-the-loop pause in LangGraph?
12. What is memory in LangGraph? How does it persist state across steps?
13. What is a LangGraph checkpointer and why is it important for production?
14. How do you implement tool calling in LangGraph?
15. What is the difference between END node and a looping edge in LangGraph?
16. How do you stream responses from a LangGraph agent?
17. What is RunnableParallel and when would you use it?
18. How do you add custom tools to a LangGraph agent?
19. What is LangSmith tracing and how do you set it up?
20. How do you handle errors and retries in a LangChain chain?

■ HARD

Hard Questions — System Design & Deep Dives

1. Design a multi-agent LangGraph workflow where one agent plans, one executes, and one validates — with error recovery.
2. Implement a LangGraph agent that can call multiple tools in parallel and merge results.
3. How would you build a self-correcting agent that critiques its own output and retries if quality is poor?
4. Explain how to implement long-term memory in LangGraph using external storage.
5. How would you migrate a LangChain v0.1 application to v0.3 LCEL architecture?
6. Design a LangGraph workflow for an AI customer support agent with escalation logic.
7. How do you implement streaming in a LangGraph multi-agent system and propagate tokens to the UI?
8. What are the performance bottlenecks in a LangChain RAG pipeline at scale and how do you address each?
9. How would you implement a LangGraph agent that maintains multiple conversation threads simultaneously?
10. Design a testing strategy for a LangGraph agent — unit tests, integration tests, and evaluation.
11. How do you implement a LangGraph subgraph and when is it architecturally appropriate?

■ Study Resources

Official Docs:

→ LangChain Python Docs — python.langchain.com

→ LangGraph Docs — langchain-ai.github.io/langgraph

→ LangSmith Docs — docs.smith.langchain.com

Best Videos/Courses:

→ YouTube: LangChain Full Playlist by Greg Kamradt (all free)

→ DeepLearning.AI: 'AI Agents in LangGraph' by Harrison Chase (free)

→ YouTube: LangGraph Tutorial by LangChain team (official channel)

Practice Projects:

→ Build: A ReAct agent that can search the web and answer questions

→ Build: LangGraph workflow with human-in-the-loop approval step

→ Build: Multi-step research agent using LangGraph + RAG

CrewAI, AutoGen & Multi-Agent Systems

Agents · Tasks · Crews · Orchestration · Communication

■ EASY

Easy Questions — Conceptual Foundations

1. What is a multi-agent system?
2. What is CrewAI and what does it simplify?
3. What is an Agent in CrewAI?
4. What is a Task in CrewAI?
5. What is a Crew in CrewAI?
6. What are the 3 main attributes of a CrewAI agent (role, goal, backstory)?
7. What is a sequential process in CrewAI?
8. What is a hierarchical process in CrewAI?
9. What is AutoGen and how does it differ from CrewAI?
10. What is an AssistantAgent in AutoGen?
11. What is a UserProxyAgent in AutoGen?
12. What does 'tool use' mean in the context of agents?
13. What is agent orchestration?
14. What is the difference between a single-agent and multi-agent system?
15. What is a manager agent in hierarchical CrewAI?
16. What is delegation in CrewAI?
17. What is the output of a CrewAI Task?
18. How do you give a tool to a CrewAI agent?
19. What is memory in CrewAI — what types are available?
20. What is kickoff() in CrewAI?

MEDIUM

Medium Questions — Implementation & Design

1. Design a CrewAI crew for automated content creation with 3 agents.
2. What is the difference between sequential and hierarchical crew process — give a use case for each.
3. How does the manager agent decide which agent to assign a task to in hierarchical mode?
4. How do you implement custom tools in CrewAI using the @tool decorator?
5. What is context passing between tasks in CrewAI and how does it work?
6. How does AutoGen handle group chat with multiple agents?
7. What is the difference between ConversableAgent and AssistantAgent in AutoGen?
8. What is a termination condition in AutoGen and why is it critical?
9. How do you integrate RAG into a CrewAI agent as a custom tool?
10. What is agent memory in CrewAI — short-term, long-term, entity memory?

11. How do you debug a CrewAI agent that is looping or not terminating?
12. What is token usage management in multi-agent systems?
13. How do you implement a validator agent that checks another agent's output?
14. What is the planning feature in CrewAI and how does it work?
15. How do you run CrewAI tasks asynchronously?
16. What are the key differences between CrewAI v0.x and v1.x?
17. How do you handle agent failures gracefully in a multi-agent pipeline?
18. What is an AgentOps integration and why is it useful for production?
19. How do you implement human-in-the-loop in a CrewAI pipeline?
20. What is a flow in CrewAI and how does it differ from a Crew?

■ HARD

Hard Questions — System Design & Deep Dives

1. Design a fully autonomous research agent system using CrewAI that: searches the web, reads papers, and produces a report — with quality validation.
2. How would you architect a multi-agent system that can handle 1000 concurrent users with different tasks?
3. Design a self-healing multi-agent system where failed agents are detected and tasks are reassigned.
4. Compare LangGraph vs CrewAI vs AutoGen for building a production code review agent — justify your choice.
5. How do you implement inter-agent communication with shared state in a distributed multi-agent system?
6. What are the security risks of autonomous agents with tool use access, and how do you mitigate them?
7. Design a multi-agent system for automated software testing — from requirement reading to test execution.
8. How would you implement budget control in a multi-agent system to prevent runaway API costs?
9. What is the A2A (Agent-to-Agent) protocol and how does it enable cross-framework agent communication?
10. Design a multi-agent debate system where 3 agents argue different positions and a judge agent evaluates.
11. How do you implement observability for a multi-agent system — what metrics would you track?

■ Study Resources

Official Docs:

→ CrewAI Docs — docs.crewai.com

→ AutoGen Docs — microsoft.github.io/autogen

→ CrewAI GitHub — github.com/crewAIInc/crewAI

Best Videos/Courses:

→ DeepLearning.AI: 'Multi AI Agent Systems with crewAI' (free, 2hrs)

→ YouTube: 'CrewAI Full Tutorial' by Brandon Hancock

→ YouTube: 'AutoGen Tutorial' by Microsoft Research

Practice Projects:

→ Build: CrewAI blog writing crew with researcher + writer + editor agents

→ Build: AutoGen code writing + testing agent pair

→ Build: Multi-agent data analysis pipeline

Prompt Engineering

Few-Shot · Chain-of-Thought · Structured Outputs · Optimization

■ EASY

Easy Questions — Conceptual Foundations

1. What is prompt engineering?
2. What is a system prompt vs a user prompt?
3. What is zero-shot prompting?
4. What is few-shot prompting? Give an example.
5. What is chain-of-thought (CoT) prompting?
6. What does 'role prompting' mean?
7. What is temperature in LLM generation and how does it affect output?
8. What is top-p (nucleus) sampling?
9. What is max_tokens and how does it affect LLM output?
10. What is a prompt template?
11. What is instruction following in LLMs?
12. What is hallucination and how can prompt design reduce it?
13. What is the difference between a completion model and a chat model?
14. What does 'be specific' mean in prompt design?
15. What is output formatting in prompts (JSON, bullet points, etc.)?
16. What is a negative example in few-shot prompting?
17. What is prompt injection and why is it a security risk?
18. What is the 'delimiters' technique in prompting?
19. What is the 'step by step' instruction and why does it help?
20. What is token counting and why does it matter for cost?

MEDIUM

Medium Questions — Implementation & Design

1. Explain Tree-of-Thought (ToT) prompting and how it differs from CoT.
2. What is self-consistency prompting and how does it improve accuracy?
3. How do you design a prompt to return structured JSON reliably?
4. What is ReAct prompting and how does it combine reasoning and acting?
5. What is Constitutional AI and how does it relate to prompt design?
6. How do you implement prompt chaining for complex multi-step tasks?
7. What is the 'meta-prompt' technique?
8. How do you handle multi-turn conversation context in prompts?
9. What is the difference between a prompt for GPT-4 vs Claude vs Gemini?
10. How do you measure and compare the quality of two different prompts?

11. What is PromptLayer and how does it help with prompt management?
12. How do you implement guardrails in prompts to prevent toxic output?
13. What is the APE (Automatic Prompt Engineer) technique?
14. How do you handle long documents that exceed context window in prompts?
15. What is grounding in prompting and how does it reduce hallucination?
16. What is the 'act as' technique and what are its limitations?
17. How do you write prompts for structured data extraction from unstructured text?
18. What is RAG prompt design — how do you format retrieved context in the prompt?
19. How do you implement prompt versioning in a production system?
20. What is the difference between system prompt and few-shot examples for task definition?

■ HARD

Hard Questions — System Design & Deep Dives

1. Design a prompt optimization system that automatically tests and improves prompts using LLM-as-judge.
2. How would you build a prompt evaluation pipeline using RAGAS-style metrics?
3. Explain DSPy — how does it automate prompt optimization and what are its core abstractions?
4. Design a jailbreak-resistant prompt system for a customer-facing AI product.
5. How do you implement dynamic few-shot selection — choosing the best examples based on the input?
6. What is Constitutional AI and how would you implement a self-critique loop in a prompt chain?
7. Design a multi-objective prompt that optimises for accuracy, brevity, and a specific tone simultaneously.
8. How do you handle adversarial prompt injection in a RAG system where user content is in the context?
9. What is the Skeleton-of-Thought technique and how does it enable parallel generation?
10. Design a systematic A/B testing framework for prompt variants in production.
11. How do you implement LLM-as-a-judge and what are its biases and limitations?

■ Study Resources

Official Docs:

→ OpenAI Prompt Engineering Guide — platform.openai.com/docs/guides/prompt-engineering

→ Anthropic Prompt Engineering Guide — docs.anthropic.com/en/docs/build-with-claude/prompt-engineering

→ Google Prompting Guide — ai.google.dev/gemini-api/docs/prompting-strategies

Best Courses:

→ DeepLearning.AI: 'ChatGPT Prompt Engineering for Developers' (free, 1hr)

→ DeepLearning.AI: 'Prompt Engineering with Llama 2' (free)

→ Learn Prompting — learnprompting.org (free, comprehensive)

Practice:

→ Build: Prompt evaluation harness that scores outputs on 3 criteria

→ Build: Dynamic few-shot selector using embedding similarity

→ Build: Structured JSON extractor with reliable output parsing

LLM APIs — OpenAI · Anthropic · Gemini

API Usage · Function Calling · Streaming · Cost Management

■ EASY

Easy Questions — Conceptual Foundations

1. What is the OpenAI Chat Completions API?
2. What are the roles in OpenAI messages: system, user, assistant?
3. What is an API key and how do you secure it?
4. What is GPT-4o and how does it differ from GPT-4?
5. What is Claude (Anthropic) and what makes it different from GPT?
6. What is Gemini and which company built it?
7. What is a token in LLM APIs?
8. How do you calculate the cost of an API call?
9. What is the difference between input tokens and output tokens in billing?
10. What is max_tokens vs context_window?
11. What is streaming in LLM APIs and why is it useful for UX?
12. What is the messages array in OpenAI API?
13. What does the 'model' parameter specify?
14. What is stop sequence in LLM generation?
15. What is logprobs and when would you use it?
16. What is the Anthropic Messages API structure?
17. How do you set up the OpenAI Python client?
18. What is a rate limit and how do you handle it?
19. What is the difference between gpt-3.5-turbo and gpt-4o-mini?
20. What is the Gemini API and how do you access it?

MEDIUM

Medium Questions — Implementation & Design

1. Implement function calling (tool use) with the OpenAI API — show the full flow.
2. What is parallel tool calling and how does OpenAI support it?
3. How does Anthropic's tool use differ from OpenAI's function calling?
4. Implement streaming with the OpenAI API in Python and process the chunks.
5. What is exponential backoff and how do you implement it for rate limit errors?
6. How do you implement a token counter before making an API call?
7. What is the Batch API in OpenAI and when should you use it (50% cost saving)?
8. How do you implement a cost tracking system for LLM API calls?
9. What is the difference between claude-3-5-sonnet and claude-3-5-haiku?
10. How do you handle multi-modal inputs (text + image) with GPT-4o?

11. What is the Responses API in OpenAI (2025) and how does it differ from Chat Completions?
12. How do you implement retry logic with jitter for OpenAI API calls?
13. What is Anthropic's prompt caching and how does it reduce costs by up to 90%?
14. How do you structure a conversation history for a multi-turn chat application?
15. What is the JSON mode in OpenAI API and how is it different from function calling?
16. How do you use the OpenAI Assistants API for file-based question answering?
17. What is Gemini's long context window (1M tokens) useful for?
18. How do you implement fallback between OpenAI and Anthropic when one fails?
19. What is LiteLLM and how does it provide a unified interface across LLM providers?
20. How do you test LLM-dependent code in unit tests (mocking the API)?

■ HARD

Hard Questions — System Design & Deep Dives

1. Design a cost-optimized LLM routing system that selects the cheapest model capable of answering a given query.
2. Implement a production-grade OpenAI client with retry, circuit breaker, rate limiting, and cost tracking.
3. How would you build a shadow-mode system that compares outputs from GPT-4 vs Claude for quality regression testing?
4. Design a streaming response architecture from LLM API through FastAPI to React frontend.
5. How do you implement a semantic cache using embeddings to avoid redundant LLM API calls?
6. What is a 'model router' and how would you train a lightweight classifier to route queries to the right LLM?
7. Design an observability system for LLM API calls — latency, cost, error rate, output quality dashboards.
8. How would you implement deterministic outputs from a stochastic LLM for critical financial calculations?
9. What is Anthropic's Constitutional AI training approach and how does it affect prompt design?
10. Design a multi-provider LLM system with load balancing, failover, and cost optimization.
11. How do you implement fine-tuning with OpenAI's fine-tuning API and when is it worth the cost?

■ Study Resources

Official Docs:

→ OpenAI API Reference — platform.openai.com/docs/api-reference

→ Anthropic API Docs — docs.anthropic.com

→ Google Gemini API — ai.google.dev/gemini-api/docs

→ LiteLLM Docs — docs.litellm.ai

Best Courses:

→ DeepLearning.AI: 'Building Systems with the ChatGPT API' (free)

→ DeepLearning.AI: 'Prompt Engineering with Claude' (free)

→ OpenAI Cookbook — cookbook.openai.com (code examples for everything)

Practice:

→ Build: Tool-calling agent that can query a REST API and database

→ Build: Streaming chat interface with FastAPI + React

→ Build: Cost tracker that logs every API call with model, tokens, cost

Python for AI — Async, APIs, Debugging

AsyncIO · FastAPI · Type Hints · Testing · Performance

EASY

Easy Questions — Conceptual Foundations

1. What is async/await in Python and why is it used?
2. What is an event loop in asyncio?
3. What is the difference between async def and def?
4. What is await used for?
5. What is asyncio.gather() and what does it do?
6. What is FastAPI and why is it popular for AI backends?
7. How do you create a basic GET endpoint in FastAPI?
8. What are Python type hints and why are they useful?
9. What is Pydantic and how does it relate to FastAPI?
10. What is a BaseModel in Pydantic?
11. What is the difference between list, dict, and tuple in Python?
12. What is a Python decorator?
13. What is a context manager (with statement)?
14. What is list comprehension? Give an example.
15. What is a generator in Python?
16. What is the difference between *args and **kwargs?
17. What is a Python dataclass?
18. What is the difference between == and is in Python?
19. What is a virtual environment and why is it used?
20. What is pip and how do you manage dependencies?

MEDIUM

Medium Questions — Implementation & Design

1. Implement an async function that makes 10 HTTP requests in parallel using asyncio.gather.
2. What is the difference between asyncio.gather() and asyncio.wait()?
3. How do you implement a FastAPI endpoint that streams an LLM response using StreamingResponse?
4. What is dependency injection in FastAPI and how does it work?
5. How do you implement background tasks in FastAPI?
6. What is Pydantic v2 and what changed from v1?
7. How do you add authentication middleware to a FastAPI app?
8. What is CORS and how do you configure it in FastAPI?
9. How do you implement proper error handling with HTTPException in FastAPI?
10. What is asyncio.Queue and when is it useful for AI pipelines?

11. How do you implement a rate limiter in Python for API calls?
12. What is the difference between threading, multiprocessing, and asyncio?
13. How do you write unit tests for an async FastAPI endpoint?
14. What is pytest and what are fixtures?
15. How do you mock an external API call in a unit test?
16. What is Python's GIL and how does it affect concurrent AI workloads?
17. How do you profile a slow Python function?
18. What is Pydantic's model_validator and when do you use it?
19. How do you implement pagination in a FastAPI endpoint?
20. What is httpx and how does it differ from requests for async code?

■ HARD

Hard Questions — System Design & Deep Dives

1. Design a FastAPI application that serves an LLM endpoint with request queuing, rate limiting, and streaming.
2. How do you implement a connection pool for async database connections in FastAPI?
3. What is the actor model and how does it apply to Python AI workloads?
4. Design a Python async pipeline that processes documents through embedding, storage, and indexing — with backpressure handling.
5. How do you implement a circuit breaker pattern in Python for external LLM API calls?
6. What are the memory management implications of loading a large embedding model in a FastAPI server with multiple workers?
7. How do you implement WebSocket endpoints in FastAPI for real-time agent streaming?
8. Design a Python test strategy for an agentic AI system — what's hard to test and how do you approach it?
9. How do you use Python's asyncio.Semaphore to limit concurrent LLM API calls?
10. What is uvicorn vs gunicorn and how do you configure FastAPI for production deployment?
11. How do you implement structured logging for an AI backend that captures request ID, model, latency, and token usage?

■ Study Resources

Official Docs:

→ FastAPI Docs — fastapi.tiangolo.com (best docs ever written)

→ Python asyncio Docs — docs.python.org/3/library/asyncio.html

→ Pydantic Docs — docs.pydantic.dev

Best Courses:

→ YouTube: 'FastAPI Full Course' by Amigoscode (free, 4hrs)

→ Real Python: asyncio tutorials — realpython.com/async-io-python

→ YouTube: 'Python Testing with pytest' by Corey Schafer

Practice:

→ Build: FastAPI backend for a RAG chatbot with streaming

→ Build: Async document processing pipeline with rate limiting

→ Build: Fully tested FastAPI app with pytest + mocked LLM calls

DynamoDB

NoSQL · Keys · Indexes · Queries · AWS Integration

■ EASY

Easy Questions — Conceptual Foundations

1. What is DynamoDB and who makes it?
2. What type of database is DynamoDB (SQL/NoSQL)?
3. What is a Partition Key in DynamoDB?
4. What is a Sort Key in DynamoDB?
5. What is a Primary Key? What are the two types?
6. What is an Item in DynamoDB (equivalent to a row)?
7. What is an Attribute in DynamoDB?
8. What is a Table in DynamoDB?
9. What is eventually consistent read vs strongly consistent read?
10. What are Read Capacity Units (RCU) and Write Capacity Units (WCU)?
11. What is On-Demand vs Provisioned billing mode?
12. What is the maximum item size in DynamoDB?
13. What is PutItem in DynamoDB?
14. What is GetItem in DynamoDB?
15. What is DeleteItem in DynamoDB?
16. What is a DynamoDB Global Secondary Index (GSI)?
17. What is the difference between Query and Scan in DynamoDB?
18. What is AWS SDK for Python (boto3)?
19. How do you connect to DynamoDB using boto3?
20. What is the DynamoDB Local for development?

MEDIUM

Medium Questions — Implementation & Design

1. Design a DynamoDB table for storing user chat history for an AI chatbot. What are your partition and sort keys?
2. When would you use a GSI vs a LSI (Local Secondary Index)?
3. What is a DynamoDB composite primary key and how does it enable range queries?
4. How do you paginate large Query results in DynamoDB?
5. What is a ConditionExpression in DynamoDB and why is it important?
6. What is an UpdateExpression and how do you increment a counter atomically?
7. What is the single-table design pattern in DynamoDB and why is it recommended?
8. How do you implement TTL (Time to Live) in DynamoDB for auto-expiring sessions?
9. What is a DynamoDB Stream and how would you use it to trigger a Lambda function?
10. How do you implement optimistic locking in DynamoDB?

11. What is a batch write and what are its limits?
12. How do you handle the 'hot partition' problem in DynamoDB?
13. What is a ProjectionExpression and how does it reduce RCU consumption?
14. Design a DynamoDB schema for storing LLM conversation sessions with TTL.
15. How does DynamoDB handle transactions (ACID) and what are the limits?
16. What is DynamoDB Accelerator (DAX) and when do you need it?
17. How would you migrate data from MySQL to DynamoDB?
18. What is the difference between FilterExpression and ConditionExpression?
19. How do you model many-to-many relationships in DynamoDB?
20. What are DynamoDB reserved words and how do you handle them in expressions?

■ HARD

Hard Questions — System Design & Deep Dives

1. Design a complete DynamoDB single-table schema for a multi-tenant AI SaaS application with users, conversations, and documents.
2. How do you implement a global leaderboard with real-time updates in DynamoDB?
3. What are the capacity planning strategies for DynamoDB at 100,000 requests per second?
4. Explain how DynamoDB's internal consistent hashing and replication works.
5. Design a DynamoDB-based session store for a distributed multi-agent AI system.
6. How do you implement full-text search on DynamoDB data (hint: DynamoDB doesn't support it natively)?
7. What is the cost comparison between DynamoDB and MongoDB Atlas at 1TB of data with 1M reads/day?
8. How do you implement an audit trail / event sourcing pattern in DynamoDB?
9. Design a DynamoDB schema that supports querying chat messages by user, by session, and by time range.
10. How do you handle DynamoDB hot partitions for a viral AI app with millions of concurrent users?
11. What is the DynamoDB Enhanced Client in Java SDK v2 and what are the Python equivalents?

■ Study Resources

Official Docs:

→ AWS DynamoDB Developer Guide — docs.aws.amazon.com/dynamodb

→ boto3 DynamoDB Docs — boto3.amazonaws.com/v1/documentation/api/latest/reference/services/dynamodb.html

→ DynamoDB Best Practices — docs.aws.amazon.com/amazondynamodb/latest/developerguide/best-practices.html

Best Courses:

→ YouTube: 'DynamoDB Full Guide' by Be A Better Dev (free)

→ AWS Skill Builder: 'Amazon DynamoDB for Serverless Architectures' (free)

→ Alex DeBrie: 'The DynamoDB Book' (paid but best resource)

Practice:

→ Build: Chat history store for a chatbot using DynamoDB + TTL

→ Build: DynamoDB single-table design for a task management app

→ Set up: DynamoDB Local and run queries without AWS costs

Docker & Containerization

Dockerfile · Containers · Compose · Networking · Production

■ EASY

Easy Questions — Conceptual Foundations

1. What is Docker and what problem does it solve?
2. What is a container and how is it different from a VM?
3. What is a Docker image?
4. What is a Dockerfile?
5. What does the FROM instruction do in a Dockerfile?
6. What does RUN do in a Dockerfile?
7. What does COPY vs ADD do in a Dockerfile?
8. What does CMD vs ENTRYPOINT do?
9. What does EXPOSE do in a Dockerfile?
10. What is docker build and what does it do?
11. What is docker run?
12. What is docker ps?
13. What is docker stop vs docker kill?
14. What is docker logs?
15. What is a Docker volume?
16. What is Docker Hub?
17. What is docker pull and docker push?
18. What is docker-compose?
19. What is a docker-compose.yml file?
20. What is the difference between docker run and docker-compose up?

MEDIUM

Medium Questions — Implementation & Design

1. Write a Dockerfile for a FastAPI + Python AI application.
2. What is a multi-stage Docker build and why does it reduce image size?
3. How do you pass environment variables securely to a Docker container?
4. What is Docker networking — bridge, host, and overlay modes?
5. How do you link two containers (e.g., FastAPI + ChromaDB) using docker-compose?
6. What is a health check in Docker and how do you configure it?
7. How do you mount a local directory as a volume for development hot-reload?
8. What is .dockerignore and why is it important?
9. How do you optimize a Python Docker image size? (layers, slim base, cache)
10. What is the difference between docker-compose v2 and v3 syntax?

11. How do you scale a service using docker-compose?
12. What is docker exec and when is it useful for debugging?
13. How do you handle secrets (API keys) in Docker without hardcoding them?
14. What is a named volume vs a bind mount?
15. How would you Dockerize a LangChain + FastAPI + ChromaDB application?
16. What is a non-root user in Docker and why is it a security best practice?
17. How do you use ARG vs ENV in a Dockerfile?
18. What is layer caching in Docker and how do you use it to speed up builds?
19. How do you use docker-compose override files for dev vs prod configs?
20. What is the difference between WORKDIR and cd in a Dockerfile?

■ HARD

Hard Questions — System Design & Deep Dives

1. Design a Docker Compose setup for a full agentic AI application: FastAPI + ChromaDB + Redis + Nginx.
2. How do you implement rolling updates for a Dockerized AI service with zero downtime?
3. What are the security hardening steps for a production Docker container?
4. How do you handle GPU access in Docker for running local LLMs (e.g., Ollama)?
5. Explain how Docker networking works at the kernel level (namespaces, cgroups).
6. Design a CI/CD pipeline using GitHub Actions that builds, tests, and pushes a Docker image.
7. How do you debug a Docker container that crashes on startup?
8. What is distroless base image and when should you use it?
9. How do you implement persistent storage for a vector database running in Docker?
10. Compare Docker Compose vs Kubernetes for deploying a 3-service AI application — when do you need K8s?
11. How do you profile and optimize memory usage of a Python AI app running in Docker?

■ Study Resources

Official Docs:

→ Docker Official Docs — docs.docker.com

→ Docker Compose Docs — docs.docker.com/compose

→ Docker Hub — hub.docker.com

Best Courses:

→ YouTube: 'Docker Tutorial for Beginners' by TechWorld with Nana (free, 3hrs)

→ YouTube: 'Docker Compose Tutorial' by TechWorld with Nana

→ Play with Docker — labs.play-with-docker.com (free hands-on)

Practice:

→ Build: Dockerize your FastAPI RAG app with docker-compose

→ Build: Multi-container setup: API + ChromaDB + Redis cache

→ Deploy: Push your Docker image to Docker Hub

Full Stack for Agentic AI

React · Next.js · TypeScript · Node.js · System Design

EASY

Easy Questions — Conceptual Foundations

1. What is the difference between React and Next.js?
2. What is Server-Side Rendering (SSR) in Next.js?
3. What is TypeScript and why is it preferred over JavaScript for large projects?
4. What is an interface vs a type in TypeScript?
5. What is Zustand and what problem does it solve?
6. What is React Query (TanStack Query) and what does it do?
7. What is Prisma ORM?
8. What is Express.js?
9. What is REST vs GraphQL?
10. What is middleware in Express.js?
11. What is JWT (JSON Web Token)?
12. What is the difference between access token and refresh token?
13. What is WebSocket and when is it used?
14. What is Server-Sent Events (SSE) and how is it used for LLM streaming?
15. What is CORS and why do you get CORS errors?
16. What is an API Gateway?
17. What is the difference between monolith and microservices?
18. What is Redis and what is it used for?
19. What is a CDN and why is it important for frontend?
20. What is environment variables and how do you use .env files in Node.js?

MEDIUM

Medium Questions — Implementation & Design

1. How do you implement SSE (Server-Sent Events) in Express.js for streaming LLM output to a React frontend?
2. What is Zustand and how does it compare to Redux for AI chat applications?
3. How do you implement optimistic UI updates in React Query for a chat interface?
4. What is Next.js App Router vs Pages Router and which should you use for an AI app?
5. How do you implement WebSocket in Node.js using Socket.io for real-time agent updates?
6. What is the BFF (Backend for Frontend) pattern and how does it apply to AI applications?
7. How do you implement authentication with JWT in a Next.js + FastAPI architecture?
8. What is Prisma schema and how do you define relations?
9. How do you implement streaming text display in React as tokens arrive from an LLM?
10. What is React Suspense and how can it help with AI loading states?

11. How would you design the frontend architecture for a multi-agent chat dashboard?
12. What is the difference between useEffect and useLayoutEffect?
13. How do you implement infinite scroll for a chat history with React Query?
14. What is ISR (Incremental Static Regeneration) in Next.js?
15. How do you implement a file upload feature for a RAG document ingestion UI?
16. What is tRPC and how does it provide type-safe APIs between Next.js and Node?
17. How do you handle race conditions in concurrent LLM API calls from the frontend?
18. What is the difference between client components and server components in Next.js 13+?
19. How do you implement a real-time agent activity feed using WebSockets?
20. What are the key performance optimization techniques for a React AI dashboard?

■ HARD

Hard Questions — System Design & Deep Dives

1. Design the complete system architecture for a production-grade AI agent platform — frontend, backend, agent layer, storage, and observability.
2. How do you implement a real-time collaborative AI workspace where multiple users see the same agent output?
3. Design a streaming architecture from LangGraph agent through FastAPI through React — handling backpressure and errors.
4. How would you build a self-hosted alternative to ChatGPT with conversation management, file uploads, and multiple models?
5. What is the strangler fig pattern and how would you use it to incrementally add AI features to an existing Node.js app?
6. Design a multi-tenant AI SaaS architecture with per-tenant rate limiting, billing, and data isolation.
7. How do you implement end-to-end type safety in a TypeScript + Next.js + FastAPI + Python project?
8. Design the database schema (SQL + DynamoDB) for a full agentic AI application with users, agents, tasks, and audit logs.
9. How would you implement a plugin system that lets users add custom tools to an AI agent from the frontend?
10. Design a frontend architecture that gracefully handles an agent taking 5+ minutes to complete a task.
11. What is your strategy for testing a full-stack AI application end-to-end — from UI interaction to LLM output?

■ Study Resources

Official Docs:

→ Next.js Docs — nextjs.org/docs

→ TypeScript Handbook — typescriptlang.org/docs/handbook

→ TanStack Query Docs — tanstack.com/query

→ Zustand Docs — zustand-demo.pmnd.rs

Best Courses:

→ YouTube: 'Next.js Full Course' by Traversy Media (free)

→ TypeScript: 'Total TypeScript' by Matt Pocock — totaltypescript.com (free basics)

→ YouTube: 'Building AI Applications' by Vercel team

Practice:

→ Build: Full-stack AI chat app — Next.js + FastAPI + LangChain with streaming

→ Build: Agent dashboard showing real-time task progress via WebSocket

→ Build: File upload + RAG pipeline with progress indicator UI